

System Requirements Specification (SRS)

Project Name: BAHub Workspace Suite
Document Version: 1.0
Status: APPROVED
Target Audience: DevOps Engineers, Security Officers, Software Architects
Date: July 2, 2026

Document Control

Version History

Version	Date	Author	Description of Changes
0.1	June 20, 2026	Technical Lead	Initial System Design Architecture
1.0	July 02, 2026	Technical Lead	baseline finalized.

1. System Architecture

BAHub is structured as a decoupled, multi-tenant single-page application (SPA) interacting with an asynchronous django backend.



Components:

1. **Frontend Client:** SPA written in TypeScript using React and Vite. Uses Tailwind CSS for styles, TanStack Query for state hydration, and Axios for network requests.
2. **Backend Services:** Django 5.x application serving REST APIs via Django REST Framework (DRF) and WebSockets via Django Channels managed by Daphne.
3. **Database Layer:** Production-grade PostgreSQL database. The application layer handles soft-deletes and filters database operations by tenant.

2. API Endpoint Directory

All responses conform to a standardized JSON envelope structure:

```
{
  "success": true,
  "data": {},
  "errors": null,
  "meta": {}
}
```

2.1 Core Authentication Endpoints

- **POST** `/api/v1/token/`
 - *Request:* { "username": "analyst", "password": "Password123" }
 - *Response:* { "access": "jwt_access_token", "refresh": "jwt_refresh_token" }
- **POST** `/api/v1/token/refresh/`
 - *Request:* { "refresh": "jwt_refresh_token" }
 - *Response:* { "access": "new_jwt_access_token" }

2.2 Requirements Management Endpoints

- **GET** `/api/v1/requirements/`
 - *Headers:* Authorization: Bearer <token>
 - *Description:* Retrieves list of requirements. Results are filtered to only return records belonging to the user's organization and projects.
- **POST** `/api/v1/requirements/`
 - *Request:* { "project": "<uuid>", "title": "Spec title", "description": "spec details", "req_type": "FUNCTIONAL", "priority": "HIGH" }
 - *Response:* Returns the created requirement with sequential `req_id` (e.g., REQ-012).

2.3 Confluence & Jira Sync Endpoints

- **POST** `/api/v1/projects/<uuid>/sync/jira/`
 - *Description:* Synchronizes project user story cards with the target Jira board via a celery task runner.
- **POST** `/api/v1/documents/<uuid>/publish/confluence/`

- *Description:* Publishes compiled markdown BRD/FRD text directly to a Confluence page using the Confluence Cloud REST API.

3. Real-Time Collaboration Protocol (WebSockets)

BAHub handles real-time collaborator tracking via a WebSocket channel layer.

- **Connection URI:** `wss://<domain>/ws/requirements/?token=<jwt_access_token>`
- **Daphne Router Mappings:** Passes the connection requests to the `RequirementConsumer` channel handler.
- **Message Payloads:**
 - **User Presence Broadcast:**

```
{
  "type": "presence",
  "user": "analyst",
  "action": "JOINED",
  "active_requirement": "REQ-002"
}
```

- **Typing Indicator Broadcast:**

```
{
  "type": "typing",
  "user": "analyst",
  "requirement_id": "REQ-002",
  "is_typing": true
}
```

4. Security & Cryptographic Implementations

4.1 Encrypted Credential Management

To integrate with third-party systems like Jira and Confluence, tenants store sensitive API keys in the database. The system encrypts these keys before database persistence.

- **Implementation Class:** Custom Django field `EncryptedCharField` inheriting from `models.CharField`.
- **Cryptographic Protocol:** Symmetrically encrypts credentials using **Fernet (AES-128 in CBC mode with HMAC-SHA256)**.
- **Key Derivation:** Dynamically derives a 32-byte URL-safe encryption key from the Django `SECRET_KEY` using SHA256.

4.2 Database Isolation (Row-Level Security)

Every core model inherits from `BaseModel`, which provides audit fields: `id` (UUID), `created_at`, `updated_at`, and `is_deleted`.

```
class BaseModelQuerySet(models.QuerySet):
    def delete(self):
        return self.update(is_deleted=True) # Soft delete override

class BaseModelManager(models.Manager):
    def get_queryset(self):
        # Exclude soft-deleted rows by default
        return BaseModelQuerySet(self.model,
            using=self._db).filter(is_deleted=False)
```

5. Performance & Scalability Constraints

- **Database Indices:** Foreign key relationships (`organization_id`, `project_id`, `requirement_id`) must use B-Tree indexing.
- **Axios Connection Cache:** The frontend must leverage TanStack Query (`staleTime: 300000`) to cache project list API responses and reduce server load.
- **Document Compilation Limits:** Markdown-to-PDF compiler actions must offload layout rendering workloads to isolated process buffers (ReportLab/WeasyPrint) to prevent memory allocation spikes.